

# CRACK GOOGLE RESEARCH INTERNSHIP 2026

Complete Roadmap · 200+ Resources · Timetable · Email Finder Tools

■  
**Top 0.1%**  
Acceptance Rate

■  
**200+**  
Curated Links

■  
**24-Week**  
Study Plan

■  
**PhD/MS/BS**  
All Levels

## ■ TABLE OF CONTENTS

|    |                                        |    |
|----|----------------------------------------|----|
| 1  | Program Overview & Eligibility         | 2  |
| 2  | Application Process Step-by-Step       | 3  |
| 3  | Research Areas & What Google Looks For | 4  |
| 4  | Coding Interview Preparation           | 5  |
| 5  | Machine Learning & Research Prep       | 7  |
| 6  | Must-Read Papers & Books               | 9  |
| 7  | GitHub Repositories to Star            | 10 |
| 8  | Reddit Communities & Threads           | 11 |
| 9  | LeetCode Strategy for Google           | 12 |
| 10 | Networking & Cold Outreach             | 13 |

|   |                                             |   |
|---|---------------------------------------------|---|
| 1 |                                             | 1 |
| 1 | CV / Resume & Research Statement Tips       | 4 |
| . |                                             |   |
| 1 |                                             | 1 |
| 2 | 24-Week Study Timetable                     | 5 |
| . |                                             |   |
| 1 |                                             | 1 |
| 3 | Tools to Find Employee Emails from LinkedIn | 8 |
| . |                                             |   |
| 1 |                                             | 2 |
| 4 | Mock Interview & Final Tips                 | 0 |
| . |                                             |   |

## 1. PROGRAM OVERVIEW & ELIGIBILITY

### What Is the Google Research Internship?

Google offers two main research tracks for students: the **PhD Research Intern** program (12-week full-time summer) and the **Student Researcher Program** (BS/MS, flexible duration). Both place you inside Google Research, DeepMind, or Google Cloud AI teams working on real large-scale problems in ML, NLP, CV, systems, and more.

#### ■ Official Links

- PhD Research Intern 2026: [Google Careers – PhD Research Intern](#)
- Student Researcher (BS/MS): [Google Careers – Student Researcher](#)
- All Research Jobs: [research.google/careers](https://research.google/careers)
- Google AI Residency: [research.google.com/residency](https://research.google.com/residency)

### Eligibility Requirements

| Track               | Degree                 | Key Requirements                                                |
|---------------------|------------------------|-----------------------------------------------------------------|
| PhD Research Intern | PhD (penultimate year) | Research experience, publications preferred, US attendance      |
| Student Researcher  | BS / MS (current)      | Strong CS fundamentals, project experience, English proficiency |
| AI Residency        | BS/MS/PhD              | Deep learning background, 12-month commitment                   |

### Domains Covered

- ■ Machine Learning & Deep Learning
- ■ Natural Language Understanding / LLMs
- ■■ Computer Vision
- ■ Data Science & Statistics
- ■■ Quantum Information Science
- ■ Human-Computer Interaction
- ■■ Algorithmic Foundations & Optimization
- ■ Computational Biology

## 2. APPLICATION PROCESS STEP-BY-STEP

### Step 1 – Polish Your Profile

- Update your CV/resume: include graduation date (MM/YY), coding languages, GPA
- Highlight research papers, arXiv preprints, conference posters
- Get your GitHub green: push projects, contribute to open source
- Update Google Scholar / Semantic Scholar profile

### Step 2 – Find the Right Role

- Browse roles at [Google Careers](#)
- Filter by 'Internship' + 'Research' + your preferred location
- Read each JD carefully – match your background to the team's research area
- Apply to MULTIPLE listings (each is a separate application)

### Step 3 – Prepare Documents

- Updated CV / Resume (PDF preferred)
- Unofficial transcript in English
- Research statement or cover letter (for PhD roles)
- List of publications / preprints / project links

### Step 4 – Submit & Follow Up

- Apply via Google Careers portal; create a profile first
- Application window: typically opens September–February each year
- PhD Summer 2026 window closed Feb 27, 2026 — watch for 2027 cycle opening ~Sep 2026
- Response time: 90+ days; no news after 3 months likely means no match this round
- Cold-email Google researchers in your area (see Section 10 & 13)

#### ■■ Pro Tip from Reddit (r/cscareerquestions)

Reaching out directly to a Google researcher in your area BEFORE applying dramatically improves callback rates. Find their email (see Section 13), send a concise 3-sentence cold email with your CV and 1 specific paper of theirs you found interesting. Ask if they are taking interns. Many researchers have discretion to fast-track candidates.

### 3. RESEARCH AREAS & WHAT GOOGLE LOOKS FOR

Google Research evaluates candidates on three dimensions: **Research Depth** (original thinking, publications), **Engineering Excellence** (clean code, scalable systems), and **Collaborative Mindset** (cross-team communication, Googleness).

#### Hot Research Areas in 2026

| Area                                      | Key Topics                                                  |
|-------------------------------------------|-------------------------------------------------------------|
| Large Language Models & Foundation Models | Transformer scaling, RLHF, RAG, multimodal LLMs             |
| AI Safety & Alignment                     | Constitutional AI, interpretability, red-teaming            |
| Efficient Deep Learning                   | Quantization, pruning, MoE, distillation, hardware-aware ML |
| Diffusion & Generative Models             | Image/video/3D generation, latent diffusion                 |
| Graph Neural Networks                     | Knowledge graphs, molecular property prediction             |
| Reinforcement Learning                    | RLHF, world models, multi-agent systems                     |
| Systems for ML                            | Distributed training, inference serving, XLA/JAX            |
| Quantum Computing                         | Quantum ML, error correction, hybrid algorithms             |

#### ■ Explore Google Research Areas

- Google Research Publications: [research.google/pubs](https://research.google/pubs)
- Google DeepMind Research: [deepmind.google/research](https://deepmind.google/research)
- Google AI Blog: [blog.research.google](https://blog.research.google)
- Google Scholar – Google Research: [Google Research on Scholar](https://scholar.google.com/citations?view_op=view_publications&hl=en&as_scd=1)

## 4. CODING INTERVIEW PREPARATION

Google research interviews include 1–2 coding rounds (45 min each on Google Docs — no IDE). The focus is on **clean code + verbal explanation**, not just the answer. You will solve 1–2 Medium/Hard LeetCode-style problems per round.

### Core DSA Topics You Must Master

| Topic                    | Key Patterns                                                     |
|--------------------------|------------------------------------------------------------------|
| Arrays & Strings         | Two pointers, sliding window, prefix sums, sorting tricks        |
| Linked Lists             | Reversal, cycle detection (Floyd), merge, two-pointer            |
| Stacks & Queues          | Monotonic stack, BFS, deque patterns                             |
| Trees & BSTs             | DFS/BFS, lowest common ancestor, serialization                   |
| Graphs                   | DFS/BFS, Dijkstra, Union-Find, topological sort, cycle detection |
| Dynamic Programming      | Memoization, tabulation, interval DP, knapsack, LCS/LIS          |
| Heaps / Priority Queues  | Top-K, merge K sorted, streaming median                          |
| Tries                    | Autocomplete, word search, IP routing                            |
| Binary Search            | Search on answer, rotated arrays, 2D binary search               |
| Bit Manipulation         | XOR tricks, power of 2, counting bits                            |
| Recursion & Backtracking | Permutations, subsets, N-Queens, Sudoku                          |
| Segment Trees / BIT      | Range queries, Fenwick tree, lazy propagation                    |

### LeetCode & Coding Platforms

- **LeetCode (Google Tag)** — Filter by Google company tag — essential: <https://leetcode.com/problemset/?companySlug=google>
- **NeetCode 150** — Best structured problem list with video explanations: <https://neetcode.io/practice>
- **Blind 75** — Classic 75 must-do problems: <https://www.teamblind.com/post/New-Year-Gift---Curated-List-of-Top-75-LeetCode-Questions-to-Save-Your-2024-oFABTWT5>
- **CSES Problem Set** — Competitive programming fundamentals: <https://cses.fi/problemset/>
- **CP-Algorithms** — Deep algorithmic references: <https://cp-algorithms.com/>
- **USACO Guide** — Structured algorithm learning path: <https://usaco.guide/>
- **AlgoExpert** — Curated Google-style problems with video: <https://algoexpert.io/>
- **InterviewBit** — Google-specific question bank: <https://www.interviewbit.com/google-interview-questions/>
- **AtCoder** — High-quality contest problems: <https://atcoder.jp/>
- **Codeforces** — Hard problem sets for advanced prep: <https://codeforces.com/>

#### ■ Google Interview Process (Coding Rounds)

- OA: 60–90 min on HackerRank / Google Forms, 2–3 medium problems
- Technical Rounds (2–3 rounds): 45 min each on Google Docs — code without IDE
- Practice coding in a plain text editor or Google Docs every day
- Always verbalize your thought process; Google values communication as much as correctness
- Time/space complexity analysis is mandatory — mention it before writing code

## 5. MACHINE LEARNING & RESEARCH PREPARATION

For research roles, expect **ML theory questions** alongside coding. You may be asked to derive gradients, explain transformer internals, or discuss your prior research in depth. Treat your CV publications as fair game for deep technical Q&A.;

### ML Theory You Must Know Cold

- Backpropagation & gradient descent variants (Adam, AdaGrad, RMSProp, LAMB)
- Transformer architecture: attention is  $O(n^2)$ , Flash Attention, MHA vs MQA
- Regularization: dropout, batch norm, layer norm, weight decay
- Loss functions: cross-entropy, focal loss, contrastive loss, triplet loss
- Bias-variance tradeoff, overfitting, early stopping, learning rate schedules
- Convolutional networks: receptive field, pooling, ResNet residuals, depthwise conv
- Generative models: VAE (ELBO), GAN (mode collapse), Diffusion (DDPM, DDIM)
- RL fundamentals: Q-learning, policy gradient, PPO, RLHF pipeline
- Evaluation metrics: BLEU, ROUGE, FID, mAP, NDCG, AUC-ROC
- Distributed training: data parallelism, model parallelism, ZeRO stages
- Quantization: INT8, FP16, BF16, GPTQ, LoRA, QLoRA fine-tuning
- Bayesian ML: Gaussian processes, variational inference, MCMC

### Online Courses & Certifications

- **Fast.ai Practical Deep Learning** — FREE — best practical DL course: <https://course.fast.ai/>
- **CS231n (Stanford CV)** — FREE — definitive computer vision course: <https://cs231n.stanford.edu/>
- **CS224N (Stanford NLP)** — FREE — NLP with deep learning: <https://web.stanford.edu/class/cs224n/>
- **CS285 (Berkeley RL)** — FREE — deep reinforcement learning: <https://rail.eecs.berkeley.edu/deeprlcourse/>
- **Deep Learning Specialization (Coursera)** — Andrew Ng's foundational series: <https://www.coursera.org/specializations/deep-learning>
- **Machine Learning by Andrew Ng** — Classic ML fundamentals: <https://www.coursera.org/learn/machine-learning>
- **Hugging Face NLP Course** — FREE — transformers & LLMs practical: <https://huggingface.co/learn/nlp-course/>
- **MIT 6.S191 Intro to DL** — FREE — annual MIT course: <http://introtodeeplearning.com/>
- **Full Stack Deep Learning** — FREE — production ML systems: <https://fullstackdeeplearning.com/>
- **Andrej Karpathy's Zero-to-Hero** — FREE — build GPT from scratch: <https://karpathy.ai/zero-to-hero.html>

### ML Interview Preparation Resources

- **ML Interview Guide (GitHub)** — Top ML interview questions & answers: <https://github.com/khangich/machine-learning-interview>
- **Chip Huyen's ML Interviews Book** — Free book — ML system design + theory: <https://huyenchip.com/ml-interviews-book/>
- **Cracking the MLE Interview (GitHub)** — ML algorithms + LeetCode in Python: <https://github.com/lizichen/crack-the-mle-interview>
- **ML Cheatsheet** — Quick reference for ML concepts: <https://ml-cheatsheet.readthedocs.io/>
- **Papers with Code** — SOTA papers with implementations: <https://paperswithcode.com/>



- **Google ML Crash Course** — Free course by Google: <https://developers.google.com/machine-learning/crash-course>
- **Made With ML** — MLOps + research-to-production guide: <https://madewithml.com/>
- **Distill.pub** — Beautifully explained ML research: <https://distill.pub/>

## 6. MUST-READ PAPERS & BOOKS

### Seminal Papers — You Must Know These

- **Attention Is All You Need (Transformer)** (2017 — Foundation of modern NLP/LLMs): <https://arxiv.org/abs/1706.03762>
- **BERT: Pre-training of Deep Bidirectional Transformers** (2018 — Google's landmark NLP model): <https://arxiv.org/abs/1810.04805>
- **GPT-3: Few-Shot Learners** (2020 — Large-scale language models): <https://arxiv.org/abs/2005.14165>
- **PaLM: Scaling Language Modeling** (2022 — Google's 540B parameter model): <https://arxiv.org/abs/2204.02311>
- **Gemini: A Family of Highly Capable Multimodal Models** (2023 — Google DeepMind's flagship): <https://arxiv.org/abs/2312.11805>
- **DDPM: Denoising Diffusion Probabilistic Models** (2020 — Foundation of diffusion models): <https://arxiv.org/abs/2006.11239>
- **LoRA: Low-Rank Adaptation of LLMs** (2021 — Efficient fine-tuning): <https://arxiv.org/abs/2106.09685>
- **Flash Attention** (2022 — IO-aware exact attention): <https://arxiv.org/abs/2205.14135>
- **Deep Residual Learning (ResNet)** (2015 — Skip connections, deep CV): <https://arxiv.org/abs/1512.03385>
- **AlphaFold 2** (2021 — Protein structure prediction): <https://www.nature.com/articles/s41586-021-03819-2>
- **Constitutional AI (Anthropic)** (2022 — AI safety training): <https://arxiv.org/abs/2212.08073>
- **Proximal Policy Optimization (PPO)** (2017 — RL algorithm behind RLHF): <https://arxiv.org/abs/1707.06347>

### Essential Books

- **Deep Learning — Goodfellow, Bengio, Courville** — FREE online — the bible of DL: <https://www.deeplearningbook.org/>
- **Hands-On ML with Scikit-Learn & TF — Géron** — Best practical ML book: <https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/>
- **Cracking the Coding Interview — Gayle McDowell** — 150 programming interview questions: <https://www.crackingthecodinginterview.com/>
- **Elements of Programming Interviews — Aziz et al.** — Advanced coding interview prep: <https://elementsofprogramminginterviews.com/>
- **Designing ML Systems — Chip Huyen** — Production ML systems design: <https://www.oreilly.com/library/view/designing-machine-learning/9781098107956/>
- **Pattern Recognition & ML — Bishop** — FREE PDF — classic probabilistic ML: <https://www.microsoft.com/en-us/research/uploads/prod/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf>
- **The Algorithm Design Manual — Skiena** — Best for algorithm intuition: <https://www.algorist.com/>
- **Introduction to Algorithms (CLRS)** — MIT textbook — theory foundation: <https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>

### Paper Discovery & Reading Tools

- **arXiv CS.LG** — Latest ML papers daily: <https://arxiv.org/list/cs.LG/recent>
- **Semantic Scholar** — AI-powered paper discovery: <https://www.semanticscholar.org/>
- **Connected Papers** — Visualize paper citation graphs: <https://www.connectedpapers.com/>
- **Paperswithcode Trending** — Trending papers with code: <https://paperswithcode.com/trending>

- **Hugging Face Papers** — Daily curated ML papers: <https://huggingface.co/papers>
- **Google Scholar Alerts** — Set alerts for your research area: <https://scholar.google.com/>
- **AI Alignment Forum** — AI safety research discussions: <https://www.alignmentforum.org/>

## 7. GITHUB REPOSITORIES TO STAR ■

### Coding Interview Repos

- **FAANG Coding Interview Questions:** <https://github.com/ombharatiya/FAANG-Coding-Interview-Questions>

600+ problems across 17 companies incl. Google — 2025/26 updated

- **Cracking MAANG Interviews:** <https://github.com/Rustam-Z/cracking-maang>

Google & Bloomberg tips, patterns, resources

- **Google Interview Prep 2026 (Gist):** <https://gist.github.com/carefree-ladka/6d1722421f9e1e46bd2dbdb5ea1b4684>

24-week roadmap, weekly tracking template

- **Tech Interview Handbook:** <https://github.com/yangshun/tech-interview-handbook>

80k+ stars — comprehensive SWE interview guide

- **Coding Interview University:** <https://github.com/jwasham/coding-interview-university>

240k+ stars — self-taught CS study plan

- **LeetCode Patterns:** <https://github.com/seanprashad/leetcode-patterns>

Categorized LeetCode problems by pattern

- **Interview Resources (aforarup):** <https://github.com/aforarup/interview>

Curated reading list incl. Google-specific resources

- **Awesome Competitive Programming:** <https://github.com/lnishan/awesome-competitive-programming>

Algorithms, CP, and interview resources

- **Company-wise LeetCode (2025):** <https://github.com/hxu296/leetcode-company-wise-problems-2022>

Google company-tagged problem CSVs

### ML & Research Repos

- **ML Interview Prep (khangich)** — Top ML system design + theory questions:

<https://github.com/khangich/machine-learning-interview>

- **Crack the MLE Interview** — ML algorithms & LeetCode in Python:

<https://github.com/lizichen/crack-the-mle-interview>

- **Deep Learning Papers Reading Roadmap** — Curated DL paper list chronologically:

<https://github.com/floodsung/Deep-Learning-Papers-Reading-Roadmap>

- **Awesome Deep Learning** — Massive list of DL tutorials and resources:

<https://github.com/ChristosChristofidis/awesome-deep-learning>

- **Transformers (HuggingFace)** — State-of-the-art NLP models: <https://github.com/huggingface/transformers>

- **nanoGPT (Karpathy)** — Minimal GPT-2 implementation — study this: <https://github.com/karpathy/nanoGPT>

- **LLM from Scratch (Raschka)** — Build LLMs step by step: <https://github.com/rasbt/LLMs-from-scratch>

- **Awesome LLM** — LLM papers, tools, and resources: <https://github.com/Hannibal046/Awesome-LLM>

- **ML YouTube Courses** — Best free ML/DL courses on YouTube: <https://github.com/dair-ai/ML-YouTube-Courses>

- **system-design-primer** — System design for research infra questions:

<https://github.com/donnemartin/system-design-primer>

- **JAX (Google)** — Google's accelerated ML framework — know this: <https://github.com/google/jax>

- **Flax (Google/DeepMind)** — Neural networks in JAX: <https://github.com/google/flax>

## 8. REDDIT COMMUNITIES & KEY THREADS

### Subreddits to Follow

- [r/cscareerquestions](#) — CS career advice, interview experiences, Google timelines
- [r/MachineLearning](#) — Research discussions, paper announcements
- [r/learnmachinelearning](#) — Beginner to advanced ML resources
- [r/artificial](#) — AI news and research discussions
- [r/deeplearning](#) — DL research and implementation help
- [r/LocalLLaMA](#) — LLM fine-tuning and deployment
- [r/datascience](#) — Data science career and technical tips
- [r/leetcode](#) — Contest strategies, interview prep tips
- [r/interviews](#) — General interview experiences
- [r/GoogIntern](#) — Google-specific intern experiences and advice
- [r/PhD](#) — PhD-specific research and internship advice
- [r/compsci](#) — CS theory and research discussions

### Must-Read Reddit Threads (Search These)

- Search: "Google Research internship experience" on [r/cscareerquestions](#)
- Search: "Google PhD intern" on [r/MachineLearning](#) for AMA threads
- Search: "cold email Google researcher" on [r/MachineLearning](#)
- Search: "Google research internship timeline 2025" for response windows
- Search: "NLP research intern Google" for domain-specific prep tips
- Search: "Google SWE intern vs research intern" for differences
- Search: "Blind 75 Google" on [r/leetcode](#) for the canonical problem list
- Search: "How to prepare ML interview Google" on [r/MachineLearning](#)

### Other Community Platforms

- **Team Blind** — Anonymous professional network — Google-specific posts:  
<https://www.teamblind.com/company/Google>
- **Glassdoor Google Intern Reviews** — Real intern interview experiences:  
[https://www.glassdoor.com/Reviews/Google-Intern-Reviews-EI\\_IE9079.0.6\\_KE7.13.htm](https://www.glassdoor.com/Reviews/Google-Intern-Reviews-EI_IE9079.0.6_KE7.13.htm)
- **LeetCode Discuss – Google** — Interview experiences with exact questions asked:  
<https://leetcode.com/discuss/interview-experience/?currentPage=1&orderBy=hot&query=:google>
- **Levels.fyi Google** — Compensation benchmarks:  
<https://www.levels.fyi/companies/google/salaries/software-engineer/>
- **Discord: Hugging Face** — ML research community: <https://discord.gg/hugging-face>
- **Discord: LAION** — Open-source AI research community: <https://discord.gg/laion>
- **Twitter/X: #MLResearch** — Follow Google Research scientists: <https://x.com/search?q=%23MLResearch>

## 9. LEETCODE STRATEGY FOR GOOGLE

Google's coding interviews are pattern-recognition exercises. The goal is NOT to memorize solutions but to internalize algorithmic patterns so you can adapt them under pressure.

### The 5-Phase LeetCode Strategy

#### Phase 1 (Weeks 1–4): Blind 75

Do all 75 problems. 2 per day. No timer. Understand each solution deeply.

Link: <https://neetcode.io/practice>

#### Phase 2 (Weeks 5–10): NeetCode 150

150 problems organized by topic. Do them in order — Arrays → DP → Graphs.

Link: <https://neetcode.io/practice>

#### Phase 3 (Weeks 11–16): Google Tag on LeetCode

Filter LeetCode by Google company tag. Focus on medium/hard problems.

Link: <https://leetcode.com/problemset/?companySlug=google>

#### Phase 4 (Weeks 17–20): LeetCode Contests

Join weekly contests. Aim to solve 2–3 problems per contest under time pressure.

Link: <https://leetcode.com/contest/>

#### Phase 5 (Weeks 21–24): Mock Interviews

Use LeetCode mock interviews, Pramp, or Interviewing.io for real simulations.

Link: <https://www.pramp.com/>

### Problem Solving Framework (Use In Every Interview)

- 1. UNDERSTAND: Read twice. Clarify ambiguities. Ask about edge cases and constraints.
- 2. EXAMPLES: Walk through 2–3 examples manually on paper/screen.
- 3. APPROACH: State your brute-force first, then optimize. Discuss trade-offs.
- 4. COMPLEXITY: State time and space complexity BEFORE writing code.
- 5. CODE: Write clean, modular code with meaningful variable names.
- 6. TEST: Walk through your code with examples. Find and fix bugs out loud.

### ■ More LeetCode Resources

- LeetCode Top Interview 150: [leetcode.com/studyplan/top-interview-150](https://leetcode.com/studyplan/top-interview-150)
- Grind 75: [techinterviewhandbook.org/grind75](https://techinterviewhandbook.org/grind75)
- LeetCode SQL 50: [For roles with data components](#)
- Pramp (Mock Interviews): [pramp.com](https://pramp.com) — Free peer mock interviews
- Interviewing.io: [interviewing.io](https://interviewing.io) — Anonymous mock interviews with FAANG engineers

## 10. NETWORKING & COLD OUTREACH

The single highest-ROI activity besides preparation: **cold-emailing Google researchers** in your area. Many Google researchers mentor interns they personally know. A warm referral can bypass the ATS entirely.

### Finding the Right Google Researcher

- Browse [research.google/people](https://research.google/people) — full list of Google researchers with areas and publications
- Search [Google Scholar](https://scholar.google.com) for papers in your area; check author affiliations
- Use [Semantic Scholar](https://semantic.scholar) to find researchers by topic
- Follow their Twitter/X account and engage with their recent work
- Check their recent papers on arXiv and deeply understand 1–2 of them
- Find their email using tools in Section 13 (Hunter.io, Apollo.io, etc.)

### Cold Email Template That Works

Subject: Research Internship Inquiry – [Your Research Area]

Dear Dr. [Name],

I am a [Year] PhD/MS student at [University] working on [specific topic]. I recently read your paper "[Paper Title]" and found your approach to [specific insight] particularly compelling — it relates closely to my own work on [brief description].

I would love to explore a research internship with your group at Google for Summer 2026. I have attached my CV and a brief research statement. Would you be open to a 15-minute chat to discuss potential projects?

Thank you for your time.

Best,

[Your Name]

[Links: CV | Google Scholar | GitHub]

### Networking Platforms

- **Google Research People Directory** — Browse all researchers by area: <https://research.google/people/>
- **DeepMind Team Page** — DeepMind team and research areas: <https://deepmind.google/about/>
- **LinkedIn – Google Research employees** — Find and connect with Google researchers: <https://www.linkedin.com/company/google/people/?keywords=researcher>
- **NeurIPS/ICML/ICLR Attendee Lists** — Researchers who present at top conferences: <https://neurips.cc/>
- **Twitter/X Research Community** — Follow @GoogleAI, @GoogleDeepMind: <https://x.com/GoogleAI>
- **ResearchGate** — Academic social network for researchers: <https://www.researchgate.net/>



## 11. CV / RESUME & RESEARCH STATEMENT

### Resume Do's & Don'ts for Google Research

#### ■ DO

- List publications/preprints with conference name and year
- Link to arXiv/GitHub for every project
- Quantify impact: 'Reduced inference latency by 40% on TPU v4'
- Include anticipated graduation date as MM/YY
- Mention frameworks: JAX, PyTorch, TensorFlow, Hugging Face
- Keep to 1–2 pages (2 for PhD students with publications)

#### ■ DON'T

- List every course you took — only relevant ones
- Use generic buzzwords without context
- Include a photo or personal details beyond name/email/links
- Skip your GitHub/Scholar profile links
- Submit without checking that all links are live and accessible
- Use fancy LaTeX formatting that breaks ATS parsers

### Resume & LaTeX Templates

- **Jake's LaTeX Resume (Overleaf)** — Most used CS resume template: <https://www.overleaf.com/latex/templates/jakes-resume/syzfjbzwjncs>
- **Google Resume Guide** — Guidance from Google's own hiring page: <https://careers.google.com/how-we-hire/>
- **Harvard CV Template** — Academic CV format for research roles: <https://ocs.fas.harvard.edu/cvs-resumes-cover-letters>
- **Resumake (open source)** — Free resume builder: <https://resumake.io/>
- **CV Compiler** — AI-powered CV review for tech jobs: <https://cvcompiler.com/>
- **Flowcv** — Clean, ATS-friendly resume builder: <https://flowcv.com/>

### Research Statement Structure (For PhD Applications)

- **Paragraph 1:** Your research area and what problem you are solving
- **Paragraph 2:** Your specific contributions — methods, results, publications
- **Paragraph 3:** How your work aligns with the Google team's research
- **Paragraph 4:** What you hope to accomplish during the internship
- Keep it to 1 page. Use concrete numbers. Avoid jargon without explanation.

## 12. 24-WEEK STUDY TIMETABLE

This timetable assumes you are starting preparation approximately 6 months before the application window opens (typically September). Adjust based on your current level. **Daily commitment: 3–4 hours minimum.**

| Week  | Phase         | Coding (LeetCode)                                                 | ML / Research                                                                       | Application Prep                                  |
|-------|---------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------|
| 1–2   | Foundation    | Arrays, Strings, Two Pointers<br>(Blind 75: first 20)             | Linear algebra, prob & stats review<br>Khan Academy / 3Blue1Brown                   | Set up LeetCode, GitHub, Scholar profiles         |
| 3–4   | Foundation    | Linked Lists, Stacks, Queues<br>(Blind 75: next 20)               | Gradient descent, backprop from scratch<br>CS231n Lectures 1–4                      | Draft CV v1; collect transcripts                  |
| 5–6   | Foundation    | Binary Search, Sliding Window<br>(Blind 75: next 20)              | CNNs, ResNet, BatchNorm<br>CS231n Lectures 5–9                                      | Research 20 Google researchers in your area       |
| 7–8   | Core DS       | Trees, BSTs, Heaps<br>(Blind 75: final 15 + extras)               | RNNs, LSTMs, Seq2Seq<br>CS224N Lectures 1–5                                         | Read 5 papers by target researchers; take notes   |
| 9–10  | Core DS       | Graphs: BFS/DFS, Dijkstra<br>NeetCode 150: Graph section          | Transformers: read 'Attention Is All You Need'<br>Andrej Karpathy's makemore series | Reach out to 10 researchers via email             |
| 11–12 | Core DS       | Dynamic Programming Part 1<br>NeetCode 150: DP section            | BERT, GPT: read papers + implement<br>Hugging Face NLP Course Ch 1–4                | Polish CV v2; write research statement draft      |
| 13–14 | Advanced      | DP Part 2 + Tries + Intervals<br>NeetCode 150 completion          | Diffusion models: DDPM paper<br>Dense retrieval, RAG systems                        | Apply to Student Researcher roles (rolling)       |
| 15–16 | Advanced      | Google-tagged problems on LeetCode<br>(Medium: 15 problems)       | Reinforcement Learning: PPO, RLHF pipeline overview<br>CS285 Lectures 1–6           | Attend 1 ML meetup or virtual conference          |
| 17–18 | Advanced      | Google-tagged Hard problems<br>(10 problems)                      | Read 3 recent Google Brain/DeepMind papers<br>Implement 1 paper from scratch        | Follow up on all cold emails; apply to PhD roles  |
| 19–20 | System Design | Mixed practice: timed 45-min sessions<br>LeetCode weekly contests | ML System Design: Chip Huyen book<br>Distributed training, feature stores           | Prepare behavioral stories (STAR format)          |
| 21–22 | Mock Phase    | Pramp mock interviews (2/week)<br>Interviewing.io sessions        | Mock research presentation prep<br>(5-min lightning talk on your research)          | Finalize research statement; get advisor feedback |
| 23    | Final Prep    | Revise all hard problems<br>Redo Blind 75 at speed                | Review all ML fundamentals<br>Make cheat sheet of key equations                     | Confirm application submissions; check status     |
| 24    | Polish        | 2–3 easy/medium per day<br>Focus on confidence                    | Light review; rest well<br>Sleep 8 hrs/night this week                              | Prepare questions to ask interviewers             |

### Daily Routine Template

| Time Block             | Activity                                                                |
|------------------------|-------------------------------------------------------------------------|
| 7:00 – 8:00 AM         | 1 LeetCode problem (timed, 45 min) + review solution                    |
| 8:00 – 8:30 AM         | Read 1 ML paper abstract/section or blog post                           |
| Evening 7:00 – 9:00 PM | Deep work: ML course lectures OR implement paper code OR mock interview |

|                       |                                                                |
|-----------------------|----------------------------------------------------------------|
| Weekend (3–4 hrs/day) | 2 LeetCode problems + 1 paper read + networking (1 cold email) |
|-----------------------|----------------------------------------------------------------|

## 13. TOOLS TO FIND EMPLOYEE EMAILS FROM LINKEDIN

### ■■■ Ethical Use Notice

Use these tools ONLY for legitimate professional outreach such as requesting research mentorship or internship inquiries. Always respect privacy, GDPR/CAN-SPAM rules, and the recipient's preference if they ask you not to contact them. Do NOT use for spam or unsolicited commercial messages.

### Top Email Finder Tools

| Tool              | Free Tier        | Best For                                                                                        | Link                         |
|-------------------|------------------|-------------------------------------------------------------------------------------------------|------------------------------|
| Hunter.io         | 25 searches/mo   | Domain search + LinkedIn handle → email. 95% accuracy for enterprise domains. Supports LinkedIn | <a href="#">accuracy</a>     |
| Apollo.io         | 10 credits/mo    | 220M+ contact database. Chrome extension for LinkedIn profiles. CRM integrations.               | <a href="#">LinkedIn</a>     |
| Skrapp.io         | 150 credits/mo   | Most generous free tier. LinkedIn-heavy workflows. Chrome + Firefox extension.                  | <a href="#">Skrapp</a>       |
| Snov.io           | 50 credits/mo    | Email finder + verifier + drip campaigns. LinkedIn prospecting plugin.                          | <a href="#">prospecting</a>  |
| RocketReach       | 5 searches/mo    | High accuracy for academic/research profiles. API available.                                    | <a href="#">h.co</a>         |
| Clearbit Connect  | Free Chrome ext. | Gmail plugin; shows LinkedIn/company info on emails you receive.                                | <a href="#">connect</a>      |
| SalesQL           | 50 credits/mo    | Real-time LinkedIn scraping. Good for finding personal emails too.                              | <a href="#">emails</a>       |
| Wiza              | 20 credits/mo    | Integrates with Sales Navigator. Exports verified emails to CSV.                                | <a href="#">emails</a>       |
| ContactOut        | 40 searches/day  | Browser extension reveals emails on LinkedIn profiles directly.                                 | <a href="#">files</a>        |
| FindThatEmail     | 50 searches/mo   | Simple name + company → email lookup. Good fallback.                                            | <a href="#">email</a>        |
| Email Permutator+ | FREE             | Chrome extension: generates all email pattern permutations for a name/domain.                   | <a href="#">permutations</a> |
| MailTester.com    | FREE             | SMTP verification — check if a guessed email is valid before sending.                           | <a href="#">valid</a>        |

### Step-by-Step: Find a Google Researcher's Email

1. Find the researcher on [research.google/people](#) or Google Scholar.
2. Get their LinkedIn URL by searching their name on [LinkedIn](#).
3. Use Hunter.io Email Finder ([hunter.io/email-finder](#)): enter name + 'google.com' as domain.
4. Try the Google pattern manually: Google emails follow *firstname@google.com* or *firstnamelastname@google.com*.
5. Verify with MailTester.com ([mail-tester.com](#)) before sending.
6. Alternative: Email via their academic institution email (often on their Google Scholar or university page).
7. Use Apollo Chrome Extension ([Install here](#)) — visit their LinkedIn and click the extension.
8. Hunter LinkedIn API: Hunter.io now supports LinkedIn handles directly via their API: [API docs](#).

### Additional Email Discovery Methods

- Check the paper's PDF: Authors sometimes list their email in the paper footer or author notes section

- **arXiv author page:** Some researchers list contact info on their arXiv profile
- **University faculty page:** If researcher is a collaborator with a university, they may have a .edu email
- **GitHub profile:** Many researchers link their email in their GitHub bio or README
- **Conference paper page:** ACL, NeurIPS, ICML papers often have author contact info
- **Research.google profile:** Click the researcher's name; some have contact links
- **Google Scholar 'follow' feature:** Set up alerts and they may reach out if they see you engaging

### ■ Chrome Extensions for LinkedIn Email Extraction

- Apollo.io Extension: [Install on Chrome](#) — visit any LinkedIn profile and get email with 1 click
- Hunter.io Extension: [Install on Chrome](#) — domain search on any website
- ContactOut: [Install on Chrome](#) — 40 free daily LinkedIn email reveals
- Snov.io Prospect: [Install on Chrome](#) — LinkedIn scraper + email verifier
- Skrapp.io: [Install on Chrome](#) — 150 free monthly credits

## 14. MOCK INTERVIEWS & FINAL TIPS

### Mock Interview Platforms

- **Pramp** — FREE — peer-to-peer mock interviews with real engineers: <https://www.pramp.com/>
- **Interviewing.io** — Anonymous mocks with FAANG engineers; record and review: <https://interviewing.io/>
- **LeetCode Mock Interviews** — LeetCode Premium — simulated company-specific rounds: <https://leetcode.com/interview/>
- **Google's Interview Warmup** — FREE — Google's own AI-powered practice tool: <https://grow.google/certificates/interview-warmup/>
- **Big Interview** — Video practice for behavioral questions: <https://biginterview.com/>
- **Exponent** — Structured practice for technical + behavioral: <https://www.tryexponent.com/>
- **Meetapro** — 1:1 coaching sessions with ex-Google employees: <https://www.meetapro.com/>

### Behavioral Interview (Googleness)

Google assesses 'Googleness' — your cultural fit, ethics, and collaborative mindset. Prepare STAR stories (Situation, Task, Action, Result) for these themes:

- Handling ambiguity or changing requirements mid-project
- A time you disagreed with a team member — how you resolved it
- Your biggest technical failure and what you learned
- Leading without authority; influencing without managing
- An innovative idea you proposed and how you drove it forward
- Balancing multiple competing priorities under a deadline
- A time you received difficult feedback and what you did with it

### Top 10 Final Tips from the Community

1. Apply to 5+ positions — each is independent and increases your chances
2. Cold-email researchers 3–6 months before the window opens, not after
3. Publish on arXiv even if it's a workshop paper — it signals research seriousness
4. Have your GitHub green with daily commits; interviewers DO check
5. Practice explaining your research to a 10-year-old — clarity is research skill #1
6. Know the research your interviewer has done — read their last 3 papers
7. For ML rounds: always discuss alternative architectures and trade-offs
8. If you don't know something, say so and reason through it — honesty beats bluffing
9. Follow up with a thank-you email within 24 hours of each interview
10. Stay active in the ML community: Kaggle, papers, open source — it builds your brand

■ **Good luck!** The Google Research Internship is one of the most competitive programs in the world — but with consistent preparation, a strong research profile, and targeted outreach, it is absolutely achievable. Start early, stay consistent, and let your research speak for itself.